

Table of Contents

Syntax.....	3
Statements.....	5
Logical and procedural interpretation.....	5
Operators.....	5
and/or.....	5
Case statements.....	5
Truth constants.....	6
A more complicated example.....	6
Additional Operators.....	7
Operators.....	7
not.....	7
if-stm.....	7
Mechanism (Procedural Interpretation).....	8
Conditional.....	8
Optimizations.....	8
Negation.....	9
Other.....	9
Constraints.....	11
Constraint Logic Programming (CLP).....	11
Arithmetics.....	11
How does it differ from common languages?.....	11
Adding strings.....	12
Coercion and Casting [semi-deprecated].....	12
Composite Data.....	13
Functors.....	13
A Practical Example.....	13
Tuple.....	13
Lists.....	13
Getting the Elements of a List.....	14
Lists as Functors.....	14
Data Structures in Cosmos.....	14
Why Tables?.....	15
Tables.....	15
Arrays.....	15
A Practical Example.....	15
Table syntax.....	16
Pseudo imperative programming.....	17
Procedural programming.....	17
while.....	17
Temporal variables.....	17
for.....	18
Generic for.....	18
Some.....	18
Static Checking.....	20
Types.....	20

Composite Types.....	20
Custom Types.....	21
Modes.....	21
The function mode.....	21
Mixing functions with relations.....	22
Object-Oriented Programming.....	23
Creating an Object.....	23
Standard Libraries.....	25
Libraries.....	25
Built-in.....	25
Iteration.....	25
Table.....	25
List.....	26
String.....	27
List.....	27
I/O.....	28
Math.....	29
Debug.....	30
Operators.....	31
Logic operators.....	31
Impure operators.....	31
Example: a tic tac toe game.....	32
How to represent the board.....	32
Drawing the Board.....	32
AI.....	34
The main functions.....	35
Adding UI.....	36
Exploring logic programming further.....	36

Syntax

Types	Data
String	"str"
Number	1, 2.5
Relation	rel(x) true;
Functor	Tuple(x)
List	[1, '2']
Table	{x=1 and y='2'}

Operators

and, or
if, when, choose
not, once

Arithmetic Operators

+ - * /
< > <= >= = !=

Conversion (casting)

str
int
num, real

Common syntax sugar.

Syntax	Output
#str	size(str)
t.x	table.get(t, 'x')
t[x]	table.get(t, x)
t.x:2	table.set(t, x, 2)
:f(2)	functor
x in l	has(l, x)

Statements

Logical and procedural interpretation

Statements may be said to have a logical and a procedural interpretation. The latter describes how our program is *run* as it's evaluated by an interpreter, aka the machine-based explanation.

The programmer may determine a statement such as `x=1 and x=2` to be *false* simply because it's mathematically inconsistent; this is the logical/declarative interpretation.

Logic/declarative interpretation

A and B: Both A and B are true.

A or B: Either A, B or both are true.

Procedural interpretation

A and B: A and B are both evaluated, in sequence.

A or B: A is evaluated. If that fails, B is evaluated.

Operators

and/or

```
> x=1 and 2=y
| x = 1 and y = 2
> x=1 or 2=x
| x = 1
| x = 2
```

As said before, **and/or** controls the order of evaluation and consequently the flow of the language.

- **and** evaluates statements sequentially. If any statement is found to be **false**, evaluation fails.
- On the other hand, **or** will only fail if all statements are found to be **false**.

But more than that, **or** introduces non-determinism to the language. A statement such as `x=1 or 2=x` may be said to be non-deterministic in that when presented to the interpreter, it may give multiple "answers".

It may be said to be short-circuiting in that if the user doesn't ask for all answers, the second statement may not need to be evaluated at all.

Case statements

A case statement is simple shorthand to **and/or**. It can also be written as *cond*.

```
//this is equivalent to ...
```

```
//(x=1 and y=1) or 2=y
case
    x = 1
    y = 1
case
    y = 2
```

Truth constants

It's possible there's no answer to be found, in which case the interpreter will give *false* as the answer.

```
> x=1 and x=2
| false
```

The two truth constants are *false* and *true*.

```
> true
| true
> false
| false
```

Any answer besides *false* is implicitly a true answer, even if the interpreter doesn't reply with *true*, i.e. *x=1* is seen as true by the interpreter.

Such statements can be used as placeholders.

```
rel p(x)
    true
```

We've not made the code for our relation *p* yet; therefore we use the statement otherwise redundant *true* as a placeholder.

A more complicated example

Let's evaluate *(x=1 or x=2) and x!=1*.

1. *x=1* is evaluated.
2. *x!=1* is then evaluated.
3. (From 2) *x=1* doesn't hold.
4. Therefore, we go back and evaluate *x=2*. (Backtracking occurs.)
5. Both *x=2* and *x!=1* now hold.

```
rel p(x)
    x=1 or x=2

rel main()
    p(x)
    x!=1
    io.writeln(x) //2
main()
```

The above code operates on a similar principle. $(x=1 \text{ or } x=2)$ is encoded by $p(x)$. Since $x \neq 1$ holds, the program will print 2.

When examining the program procedurally, we see that a program may go back to an early or-stm to ensure correctness. This is called backtracking and it's said that an or-stm creates a *choice-point*.

Additional Operators

Operators

Adding to our repertoire, we have *if* and *not*.

These operators are meant to represent statements in the form,

If X , then Y .

If X , then Y , else Z .

not X .

Which are common in logic and show up in practical programming.

It's possible for an if-stm to be accompanied by an else-clause. An else-clause is evaluated when the condition doesn't hold.

These are examples of their uses,

not

```
> not 2=2
| false
> not 1=2
| true
> not x=2
| x!=2
> not true
| false
```

Logical interpretation

The negation of A is false when A holds, and true when A doesn't.

if-stm

```
> if(x=1) a=2;
| x=2 and a=2
| true
> if(x=1) a=2 else a=1;
| x=2 and a=2
| a=1
> if(false) x=2 else x=1;
| x=1
```

```

if(true)
    print('condition is true')
else
    print('condition is not true')

```

Logical interpretation

`if(A) B else C; <==> (A and B) or (not A and C)`

`if(A) B; <==> (not A) or B`

Mechanism (Procedural Interpretation)

A straightforward implementation of `if` and `not` akin to the one in procedural or functional languages is available through use of modes (see [Static-Checks](#)).

Therefore, the discussion below concerns the *relational* use of such operators.

Some additional attention must be taken to their working, when compared to `and/or`.

A relational implementation of them in Cosmos is experimental. It's up to change and we may accept suggestions on the matter.

Conditional

The *reif* or *reified if* is applied when the condition is a simple equality with atomic operands. The optimization avoid unnecessary use of non-determinism.[1] As of now,

- A `i=0, x=t.y` or `l=[x]` condition will be optimized. `x<0` or `x=p( )` may not work with the optimization.[2]
- Failing that,

`if(A) B else C;` is expanded to `(A and B) or (not A and C).`

`if(A) B;` is expanded to `(A and B) or (not A).`

See an explanation on `not` below.

[1] See indexing dif/2.

[2] The latter needs to *access* an element 'y', the former is syntax for `p(x)`.

Optimizations

See `fact` below.

```

rel fact(x,y)
    if(x=0)
        y=1
    else
        y=x*fact(x-1)

```

Negation

1. If the statement is simple inequality, it's reversed, e.g. `not x<=1` becomes $x>1$ and `not x=1` becomes $x\neq 1$.
2. If not, a compiler error will be given.

Why?

After some testing, we arrived to the following conclusion.

Issuing a compiler error is often beneficial to the user. A lousy implementation of `not` may not do what the user intended it to do. In such cases, an error will avoid possible issues from negation-by-delayal[3]. It's likely that, if the user comes from a procedural background, they really intended to use the procedural approach, so once again an error is helpful. Seeing an error, the user then simply looks for the procedural approach, or whatever approach it is they're looking for.

Therefore, an error is issued if relational `not` hasn't been fully implemented for a given statement.

[3] The approach used by MU-Prolog. *not* $p(x)$ is delayed until x is ground. "*If a MU-PROLOG program returns some answer, you can be confident that it is correct.*" - An Introduction to MU-PROLOG.

Other

functions

Typically, in a procedural language, a straightforward implementation of `not/if` is possible by not allowing non-ground variables. Once again, this is covered by [modes](#).

```
function q(x)
    ...
    if(not p(x))
        ...
q(2)
```

In summary, the programmer might encase use of *not* inside a function. A function will use procedural mechanisms for both *not* and *if*. The compiler will ideally check if parameters are ground.

manual implementation

It goes without saying but logic programmers have the option of implementing negative statements manually.

```
rel is_two(x)
    x=2
rel non_two(x)
    x!=2
```

A relation *non_two* is implemented manually as a negation of *is_two*.

The *when* operator provides a simple expansion. `when(A) B else C`; is expanded to `(A and B) or C`.


```
when(x=2)
  y=2
else
  non_two(x)
```

`when` will not attempt any optimizations, and is simply a different way of writing `case`, so it can be used whenever you want to manually provide the negation, or none is necessary.

Constraints

Constraint Logic Programming (CLP)

Arithmetic statements are converted to constraints when possible.

```
//this program fails
//as the statement/constraint 'x>5' is contradicted later by 'x=1'
x>5
x=1
```

In general, a statement or assertion in Cosmos holds true until proven wrong. In a sense, it's the same as what relational programming already does with equality-except it's applied to the rest of arithmetic operators.

- $x > 5$ holds true until a statement like $x = 1$ disproves it (x is instantiated to a value below 5).
- $x = 1$ holds true until $x = 2$ or $x > 1$ disproves it (x is either instantiated to 2 or above 1).

Logic interpretation

An arithmetic statement or definition holds true until proven otherwise.

Procedural interpretation

The constraint system takes care to store relevant statements as *constraints*. If the system sees that a stored constraint was contradicted, evaluation may fail.

Arithmetics

Cosmos uses both floating-point and constraint arithmetic as its default.

A lot of constraint systems would not work well as a general-purpose arithmetic. For example, CLP(FD) is limited to integers. Thus, Cosmos uses CLP for Reals[1].

The use of floating-point arithmetic works for a high-level logic-based scripting language and we avoid the more pressing issue of mixing constraint systems.

[1] As far as we know, CLP for Reals goes back to Prolog III. See

<http://prolog-heritage.org/en/ph30.html>. Many advances in LP that we use were already proposed in Prolog III and NU-Prolog.

How does it differ from common languages?

In a regular language, $x > 5$ would fail if x is not instantiated.

If x is instantiated, it should behave the same as any non-CLP language. Thus, our exploration of CLP arithmetics.

An $x=1-y$ expression is automatically interpreted to be a constraint. An user does not need to load any special CLP library in order to avoid using impure arithmetic. A relation in our language is pure by default.

Adding strings

Addition is not solely reserved for numbers. It may be used to concatenate strings, lists and arrays.

```
s='1'+ 'b'
print(s) // '1b'
print([1]+[2]) //[1,2]
```

The operator `+` depends on type. This is not true for the other arithmetic operators.

Coercion and Casting [semi-deprecated]

The special functions *num* and *str* will forcefully turn an expression into a number or string. This is not entirely unlike *casting*. As this is generally done automatically, new code may not need to make use of these functions.

If it is not possible to do so, a runtime error is given.

```
print(num(x+1)) //this may lead to a runtime error, e.g. if x is not instantiated
```

The function *int* converts any floating-points to integers. This is usually done for interaction with host languages.

Composite Data

Functors

We start with a declaration.

```
functor(F, Functor)
```

By using the special relation *functor*, we declare that F is a functor, or rather, that it'll be used to make functors.

```
F(1, 2) = F(1, a)
print(a) //2
```

Functors are a kind of composite data. Some may call it a *tuple*.

If 1 or 'hello' is a single value, $F(1, 'hello')$ is a value made from combining both.

Since the two functors we made were said to be equal to each other, we could conclude that $a=2$.

A Practical Example

Let's say we want to represent a person in our program. We could just make a functor **Person**.

```
functor(Person, Functor)
```

We can now make our first person.

```
bob = Person('bob', 23)
```

We have made the convention that the first two fields of **Person** stand for name and age, though if needed we could have included more fields. This allows us to make programs about one or more persons!

This is not the only way we could represent a person, as will be shown later.

Tuple

If we simply want to group things together, any functor is enough. We may call a functor for general use T standing for *tuple*.

```
> T(x,1) = T('x', y)
| x='x' and y=1
```

Lists

A list is another kind of composite data. Let's say we want a list of things. This can easily be done with the syntax,

```
l = [1, 2, 3]
```

We've made a list with the values 1, 2 and 3.

There are operations we can make on lists. We can add, subtract or search for elements within it. These can be found in module `list`.

```
l2 = list.push(l, 55)
io.writeln(l)    //[1, 2, 3]
io.writeln(l2)   //[1, 2, 3, 55]
```

If we *push* value 55 into `l`, we'll get a new list `l2` with elements.

Getting the Elements of a List

A common operation is to get the *first* element in a list, along with the *rest*- the remaining, which itself is a new list. These are sometimes called the head and tail of a list.

```
l = [1,2,3]
list.first(l, head) //head is 1
list.rest(l, tail)  //tail is [2, 3]
```

This is a common pattern in declarative programming, and as such there's even a special syntax for it.

```
l = [1,2,3]
l=[head|tail] //head is 1 and tail is [2, 3]
```

The term `[x|y]` is very different from `[x, y]`- the latter is a list with two elements. The former is any list.

Lists as Functors

Lists are implemented in terms of the functor `Cons`. These lists are identical:

```
l = [1, 2]
l = Cons(1, Cons(2, Cons))
```

Data Structures in Cosmos

The default data structures provided by the language have been chosen to address a common pattern. Declarative languages, specially old ones, traditionally focus on 'functor-like data' (lists and trees) over 'table-like data' (dictionaries, arrays, objects). Conversely, imperative languages will ignore lists and trees, providing an array or dictionary.

The above is only a general observation. It doesn't mean to imply declarative languages have no dictionaries. The bias is more pronounced when looking at old languages. An old functional language might often specialize on lists, then fail to offer dedicated syntax for arrays. An imperative language will easily provide arrays, etc.

The goal with our main data structures was to avoid such a bias and provide support for any style of programming. Functors offer a 'functional' and tables offer an 'imperative' style of programming, so-to speak.

Nevertheless, the programmer can choose to implement their own structures (whether using tables/functors or the host language directly) if they need more specialization as user-made libraries. This is important for those who may prefer a more comprehensive approach.

Why Tables?

Tables are much like *JavaScript Objects* and *Lua tables* of the same name.

Tables in prototypal languages can fulfill a number of uses and are highly useful,

- They can simply be used as dictionaries.
- They can be used as modules, arrays or objects.
- They have been the basis of description languages (aka JSON).

Cosmos is a declarative language, however, and differs in that such structures are immutable.

Tables

Tables (also known as maps, dictionaries, etc.) are structures that map keys to values.

```
Table t = {x=1 and y=2}
table.set(t, 'a', 1, t2)

print(t) //{ 'x': 1, 'y': 2 }
print(t2) //{ 'x': 1, 'y': 2, 'a': 1 }
```

Arrays

Arrays are tables that use numbers as keys.

```
t = {1, 'a'}
print(t) //{0: 1, 1: 'a'}
print(t[1]) //'a'
```

A Practical Example

Let's represent a person using tables.

```
person = {
    name='bob'
    age=23
}
```

We can now access the fields of our person with the following syntax,

```
//access properties
print(person.age) //23
```

```
print(person['age']) //23
```

Table syntax

Tables have the same syntax as the rest of the language, so that whitespace rules still applies.

```
t = {  
    rel p(x)  
      x=2  
    y=1  
}
```

This syntax is specially useful when making modules or objects (in which case p is a method and y is a property).

It's possible to, for example, move relations to or out of a table at any point.

This is shorthand for,

```
t = {'p'=rel(x) x=2; and 'y'=1}
```

What we've done is,

1. Assign an anonymous relation to field ' p ',
2. Assign 1 to field ' y '.

Note that keys are automatically converted to *strings*.

Pseudo imperative programming

Procedural programming

The aim of our following constructs is naturally to allow the user to program in a style that's natural in procedural programming, although in a self-contained manner. The procedural interpretation of our *while* statement thus clearly mimics the *while* statement of procedural languages. This allows the user to try a variety of programming styles even while keeping the code's logic or functional properties.

Imperative programming: code is a recipe of instructions. The interpreter will execute these instructions in order. Variables are cells that can be set to different values.

Procedural programming: in addition to the above, code may contain control structures such as 'if' or 'while' that change the flow of execution.

while

Here is an example of a *while* construct.

```
init i=1
while(i<6)
    print(i)//1,2,3,4,5
    next i=1+i
print(i)//6
```

The code will successively write every integer from 1 to 6.

Logical interpretation

while(A) B: If the condition A is true, then B holds, and remains so while A holds. Conversely, B is true until A stops holding.

Procedural interpretation

The condition A is tried; if it holds, execution proceeds to the code in B, if not, leave the *while* statement. This is repeated until the condition doesn't hold.

Temporal variables

We'd like to call our indexing variable *i* a *temporal variable*, for lack of a better term, so as to give it a better declarative reading. In this reading, a temporal variable is a variable for which the value changes over time.

These expressions are relevant for a temporal variable.

init i: the initial value of the variable.

next i: the value it'll hold in the next loop or iteration.

When inside the loop, the term i by itself is the current value. Outside, it is the final value of the variable.

In our example, the variable is continually increased, meaning that the condition eventually stops holding.

for

The classic for-statement is simply an extension of the *while* statement. It's perhaps not necessary, but we thought best to include it.

```
for(init i=1; i<6; next i=1+i;)
    print(i)//1,2,3,4,5
print(i)//6
```

The only difference is that we made the parts of code concerned with iteration clear by placing them separate from the body; our variable iterates from 1 to 5, increasing by 1 each time, and it is written.

Generic for

A simple way to do so is also by using a generic mechanism.

```
range=math.range
for(x in range(1,6))
    print(x)//1,2,3,4,5
```

The relation *range* is used to iterate through 1 to 6.

```
for(x in [1,2,3])
    print(x)//1,2,3
```

We may also incidentally move through any data structure in such a way, as long as an iteration mechanism is provided for them.

Some

It's curious that we started by mimicking procedural languages, and arrived at a way of expressing what are essentially logic idioms. Consider,

```
for(x in l)
    human(x)

some(x in l)
    human(x)
```

This expresses that *all x are human* or *some x are human*, respectively.

Operator

Idiom

for "*For all x, x is y*" (All X are Y.)

some "*There is at least one x, such that x is y*" (Some X are Y.)

Some examples may illustrate the idea.

```
>for(x in [1,2,3]) odd(x)
| false
>some(x in [1,2,3]) odd(x)
| true
>for(x in {1,1,1}) x=1
| true
>some(x in {2,1,1}) x=1
| true
```

Static Checking

Cosmos is statically-typed.

Before the language is run, programs are to be checked for errors in the compiling stage. This is different from runtime errors that happen when the program is running.

Types

What should happen when we compare values of different types?

```
1= 'b'
```

A possible solution is to make the comparison yield *false*. This is not wrong as 1 and 'b' are different values.

More often than not, though, comparing different types is a symptom of an error and not what the programmer wants. As such, you may get,

```
> x=1 and x='b'  
| CosmosError: cannot unify types 'Number' and 'String'
```

The same can happen with relations.

```
double(2, 'y')  
| CosmosError: calling relation "double" of type 'Relation Number Number' as  
'Relation Number String'
```

Once you understand that `Relation Number Number` is a relation that takes two numbers, this error is actually very readable.

Thanks to latest advances in FP, the compiler is able to detect types even when not explicitly typed by the user.

This allows for a very free style. The programmer may write very simple code, e.g. `x=1`, and the compiler can infer that `x` is a number. It would be more verbose to write `Number x=1`.

Composite Types

You may freely make what we call a composite type with functors.

```
Functor String Number f2 = F('apple', 2)
```

Or,

```
functor(F, Functor String Number)  
f2 = F('apple', 2)
```

As previously shown, relations are no different. `Relation Number Number` is written as a composite type.

Types	Data
Functor	a functor
Functor Any Any	a functor with two unspecified arguments
Relation Number String	a relation with number and string arguments
Number	a number, e.g. 1
Table	a table, e.g. {x=1 and y='2'}
Note that <code>Functor Any Any</code> refers to a functor with two parameters, while <code>Functor</code> is any functor whatsoever.	

This is intuitive, though. If someone were to say, "I need a functor!" it clearly might be any functor, while if someone bothers to say "I need a 'functor number number'" they're making such an oddly specific statement that we may assume a 'functor number number number' will not suffice.

Custom Types

...

Modes

Modes are like types for logic programming. Cosmos™ provides two modes, *rel* and function (or *fun*). The latter essentially lets the user make a function.

This is simplified compared to modes in other LP languages as there's an intuitive and practical focus. It's based on the assumption that the programmer will either follow a pure logical style (in which case they may use *rel*) or needs to fall back from that (in which case *function* is enough). For simplicity, it ignores other, more specific cases, although more modes can be added if the need arises.

In Cosmos, a function is essentially a relation that's constrained and has limited behavior. A relation built with *rel* is unconstrained.

This is designed so that you may at any time switch a function to a relation or vice versa by simply switching modes. Simply having the *rel* mode should mean it's a pure relation with a logical interpretation, although you may need to check if it's the one intended.

The function mode

A function,

- Only yields one answer.[1]
- Doesn't yield *false*, i.e. no answer.

The main change in using the function mode is that the *conditional* itself changes. A function is meant only to receive input parameters and produce output from them, so that even a procedural if-else suffices. It takes the first answer and moves to the body, or if the condition fails move to the else clause.

[1] Note: This is not completely enforced by the Cosmos™ compiler, as of yet. As such, we ask to have some caution when using the function mode. If used incorrectly, it can lead to unsound code.

Mixing functions with relations

Having a *main* function as the entry point to your program is often beneficial.

```
rel multiple(x,y)
    y=x or multiple(x+x,y)

rel odd(x)
    x%1=0

function main()
    print('type a number')
    io.write('> ')
    io.read(x)
    if(multiple(x,y) and odd(y))
        io.write('An odd multiple of '+x+'is: ')
        io.write(y)
main()
```

A program typically should not fail or return multiple results. This is ensured by having *main* as the entry point.

Because it is inside a *function*, the conditional will pick only one answer for the program.

Regardless, *multiple* and *odd* are relations. There may be many odd multiples of a given number. As such, the program exhibits relational behavior.

The function *main* picks the first number that's an odd multiple of *x*.

It's thus perfectly possible to have a program that mixes functions and relations.

Object-Oriented Programming

Our approach to objects highly mirrors that found in prototypal languages where an *object* is literally a *table* and can in some ways be used similarly.

In particular,

- Objects are opt-in.
- Existence of objects does not conflict with usage of tables as a dictionary.

It was our intent that the programmer does not have to use the paradigm and that tables can still be used separately.

```
require('object', Object)
update=Object.update

Ship={
    x=1
    y=1
    rel move(this,x,y,this2)
        update(this,{ 'x'=x and 'y'=y },this2) //this2=this+{'x'=x and 'y'=y}
}

ship=Object.create(Ship)
print(ship.y) //1
shipAfterMoving=ship.move(1,2)
print(shipAfterMoving.y) //2
```

The method `Object.create` essentially marks a table as an 'object'. Typically, a parameter `this` or `self` refers to the object itself and is hidden in further calls.

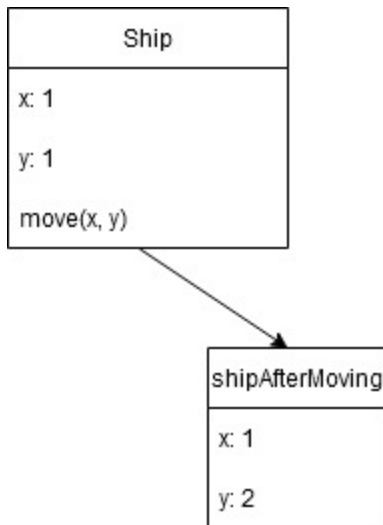
Creating an Object

Much like logic programming, object-oriented programming is often accompanied by an intuition. There's something the code tries to represent and a mechanical approach by which it does so.

Intuition

A common intuition in object-oriented programming is to say a *class* represents the ideal concept of a ship while *objects* make up individual ships.

While the concept of a Ship is unchangeable, individual ships can change and hold distinct properties.



A given ship may deviate from the ideal ship as well as other ships. An object *ship1* may be in a different position than *ship2*.

In modeling our class, we decided on its related properties--namely, *x* and *y* coordinates indicating its position, and associated methods--a *move* relation that changes its coordinates. A ship is able to move through the oceans.

```
ship=Object.create(Ship)
```

A ship is then created using the *create* relation. *Ship* may be called its class or prototype.

Procedural Interpretation

It's common in prototype-based programming to simply use a *table* or similar data-structure for the object and/or classes[1]. What happens then is,

1. `create` makes a new table `ship`.
2. The table `ship` is marked as an "object".
3. `Ship` is established as the prototype of `ship`.
4. If the programmer tries to access a property of an object, it first looks in the object itself, and if it's not found, it looks into its prototype.
5. The first parameter of an object call is hidden, hence `ship2=ship.move(ship, 1, 2)` is shortened to `ship2=ship.move(1, 2)`.

In our example, `shipAfterMoving.y` is immediately found to be 2. However, `ship.move` requires lookup as `move` only exists in the prototype.

[1] Thus, classes don't technically exist. What we have is objects that may be described as "class-like". See *Self: The Power of Simplicity*.

Standard Libraries

Libraries

Built-in

These relations are built-in to the language and don't belong to a module.

print

functor

require

Iteration

Iteration uses a table with the following relations.

iter.Relation Any Any

iter(collection, it) turns a collection into an iterable it.

next.Relation Any Any Any Any Any

next(it, it2, key, value, i) gets key-value from an iterable it and provides the next iterable it2. i is 0 on an empty collection, otherwise 1.

Table

concat.Relation Any Any Any

fold.Relation Any Any Any Any

get.Relation Any Any Any

has.Relation Any Any

imap.Relation Any Any Any

iter.Relation Any Any

join.Relation Any Any Any

map.Relation Any Any Any

new.Relation Any

next.Relation Any Any Any Any Number

set.Relation Any Any Any Any

toList.Relation Any Any
toListKeys.Relation Any Any
toListValues.Relation Any Any
update.Any

List

at.Relation Any Number Any
concat.Relation Any Any Any
each.Relation Any (Relation Any Any) Any
eachIndex.Relation Functor Any Any Functor
every.Relation Any (Relation Any)
filter.Relation Any (Relation Any) Any
find.Relation Any Any Number
findOnce.Relation Any Any Number
first.Relation Any Any
fold.Relation Any Any Any Any
forall.Relation Any (Relation Any)
get.Relation Any Number Any
has.Relation Any Any
has2.Relation Any Any
iterate.Relation Any Number Any Any
join.Relation Any String String
last.Relation Any Any
length.Relation Any Number
map.Relation Any (Relation Any Any) Any
pop.Relation Any Any
push.Relation Any Any Any
remove.Relation Any Any Any
removeAll.Relation Any (Relation Any) Any
removeIndex.Relation Any Any Any

rest.Relation Any Any

reverse.Relation Functor Functor

size.Relation Any Number

sub.Relation Any Any Any

unique.Relation Any Any Any

String

_add.Relation Any Any Any

at.Relation Any Any Any

code.Relation String Any

concat.Relation String String String

find.Relation String String Any

findIndex.Relation String String Any Any

first.Relation String String

get.Relation String Any String

has.Relation String String

length.Relation String Any

lessOrEqual.Relation String String

lower.Relation String String

rest.Relation String String

size.Relation String Any

slice.Relation String Any Any String

split.Relation String String Any

toCodes.Relation String Functor

upper.Relation String String

List

Table {at.Relation Any Any Any

concat.Relation Any Any Any

each.Relation Any (Relation Any Any) Any

eachIndex.Relation Functor Any Any Functor
every.Relation Functor (Relation Any)
filter.Relation Functor (Relation Any) Functor
find.Relation Any Any Number
findOnce.Relation Any Any Number
first.Relation Any Any
fold.Relation Functor Any Any Any
forall.Relation Functor (Relation Any)
has.Relation Any Any
has2.Relation Any Any
iterate.Relation Any Number Any Any
join.Relation Any String String
last.Relation Any Any
length.Relation Any Any
map.Relation Functor (Relation Any Any) Functor
pop.Relation Any Any
push.Relation Any Any Any
remove.Relation Any Any Any
removeAll.Relation Functor (Relation Any) Functor
removeIndex.Relation Any Any Any
reverse.Relation Functor Functor
size.Relation Any Any}

I/O

Input and output utilities. Use `require('io')`.

close.Relation File
exists.Relation String
fileReadChar.Relation File String
fileReadLine.Relation File String
fread.Relation File String

fwrite.Relation File Any

open.Relation String String Any

openBinary.Relation String String Any

read.Relation String

readFile.Relation String String

Ever wanted to simply get an entire file as a string? Because sometimes all you need is to get the text of a file as a string. No need to pass through file objects, synchronous-related documentation, etc. Just get a file as a string. Doesn't it sound great for scripting? Yet this incredibly useful function is not provided in the core of many languages and the user is left to custom implementation. It's there if you use Cosmos.

Perhaps one of the language's great gems, it takes the name of a file and returns the content as a string.

```
s=io.readFile('in.txt')
```

write.Relation Any

write16.Relation Any Any

write32.Relation Any Any

write8.Relation Any Any

writeFormat.Relation Any

writeToFile.Relation String String

writeln.Relation Any

Math

abs.Relation Integer Integer

add.Relation Any Any Any

ceil.Relation Real Integer

dec.Relation Any Any

div.Relation Any Any Any

floor.Relation Real Integer

inc.Relation Any Any

integerToReal.Relation Integer Real

integerToString.Relation Integer String

max.Relation Real Integer

min.Relation Real Integer

mul.Relation Any Any Any

random.Relation Real

realToInteger.Relation Real Real

realToString.Relation Real String

sqrt.Relation Real Real

stringToNumber.Relation Any Any

sub.Relation Any Any Any

Debug

Operators

Logic operators

if

`if(A) B else C;`

1. A has the form ``x=y`` and both operands are atomic,
 - It uses the reif optimization to avoid creating choice-points.
2.
 - It is expanded to ``(A and B) or (not A and C)``.

`if(A) B;`

- Similar to above. It applies the reif optimization if possible, otherwise expands to ``if(A) B else not A;``.

not

1. The statement is an inequality,
 - It's reversed, e.g. ``not x<=1`` expands to `_x>1_`, ``not x=1`` expands to `_x!=1_`.
2. If not,
 - A compiler error will be given.

when

`when(A) B else C;`

- Expands to ``(A and B) or C``.

if (function)

Compiles to `choose`.

not (function)

Negation-as-failure. If A fails, not A succeeds, otherwise fails.

Impure operators

once

If A fails, once A fails. Otherwise, succeed once.

choose

Procedural conditional.

`choose(A) B else C;`

- If once A succeeds, evaluates B, otherwise evaluate C.

cut

Prolog's cut.

Example: a tic tac toe game

As an example, we'll implement a simple tic-tac-toe game with user input.

How to represent the board

A few choices are,

1. An array.
2. A list.
3. An array of arrays, etc.

We choose a single array with *get/set* relations to modify and access.

Since this is a game played on a 3x3 board, the choice won't heavily impact the performance, and you can change later by changing its *get/set* relations.

```
range=math.range

_board={' ',' ',' '',' ',' '',' ',' '',' ',' '',' ',' '',' ',' '',' '}

rel set(o,o2,x,y,e)
    i=x+(y*3)
    table.set(o,i,e,o2)

rel get(o,x,y,e)
    i=x+(y*3)
    e=o[i]

rel empty(board,x,y)
    get(board,x,y,' ')
```

As we use a single array, a calculation is made to convert coordinates like $x=0, y=1$ to the element numbered 3 in the array.

If you get the string ' ', naturally it means the board is empty.

Drawing the Board

```
write=io.write
rel draw(board)
    init i=0
    while(i<3)
        init j=0
        write('|')
        while(j<3)
            tile=get(board,j,i)
            write(tile)
            write('|')
            next j=j+1
        write('\n')
        next i=i+1
```

The board is drawn in a typical procedural fashion using while-statements, so that it draws tiles and columns from 0 to 2. Tiles might be either ' ', 'X' or 'O'.

The idiom `write=io.write` gets the relation `write` directly from module `io`.

```
rel full(board)
  for(x in board)
    x!=' '

rel win(board,piece)
  case
    some(col in range(0,2))
      get(board,0,col,piece)
      get(board,1,col,piece)
      get(board,2,col,piece)
  case
    some(line in range(0,2))
      get(board,line,0,piece)
      get(board,line,1,piece)
      get(board,line,2,piece)
```

It's easy to see if the board is full (aka. every square has been filled by a piece) by checking that it has no empty tile.

We may also see if a board is won by checking that a full line or column has been filled. We are back to declarative programming and we may read the whole as,

It's a win if either,
There's `_some_` column such that every tile in it has a piece,
There's `_some_` line such that every tile in it has a piece,

The same is true for `full`,

It's a full board when,
Every tile is not empty.

This leaves checking diagonals.

```
case //check the 0-2, 1-1, 2-2 diagonal
  for(pos in range(0,2))
    get(board,pos,pos,piece)
case //check the remaining
  get(board,2,0,piece)
  get(board,1,1,piece)
  get(board,0,2,piece)
piece!=' '

```

We may end with the declaration that `piece != ' '`. That is, we'll not declare a win for a player if the piece found is the empty tile.

However, in this example we call `win` with an argument for `piece`, so that in any case it won't spot the empty tile.

AI

```
//ai
//_(board,x,y)
rel leftmost(x)
    x=0 or x=1 or x=2

rel _default(board,x,y)
    leftmost(x) and leftmost(y)
    empty(board,x,y)

function _player(board,x,y)
    print('> input line:')
    y=math.stringToNumber(io.read())-1
    print('> input column:')
    x=math.stringToNumber(io.read())-1
    if(not empty(board,x,y))//print([x,y])
        throw('invalid move')
```

Here we have the chance to explore AI programming, although on a basic level.

the AI interface

We decide that an AI function or predicate may look at a `board` and decides on a `x, y` position which is its move. That's the AI's interface. Any function or predicate with `board` and position as parameters may thus be an AI.

the simplest AI

Our *default* AI is maybe the simplest. It's actually simply a declarative logic-style definition of what a move is and it works because this is a declarative logic language.

```
x is either 0, 1 or 2.
y is either 0, 1 or 2.
The position x,y is empty.
```

It states that `x/y` are either 0, 1 or 2 and that it's possible to move there. It'll therefore pick the first available move.

This is of course not the greatest AI, and the programmer is invited to improve on it.

a random AI

An AI that picks an available move randomly. A simple random number generator is provided by `math.random`.

an optimal AI

An optimal AI could be done by looking at all possible choices one by one and picking an optimal move, i.e. one that wins the game, or draws if that's not possible. It's possible to do this in a tic-tac-toe game, but it's too resource-intensive for a more complex one. It's not necessarily good to make such an AI as it'll give the player no chance to win.

the player

The player is asked what move to make. The player inputs a move in typical 1-indexing, and this is converted to 0-indexing for the program.

The main functions

```
rel move(board, p, board2)
    p.ai(board, x, y)
    set(board, board2, x, y, p.piece)

function game(board, p1, p2, turn)
    if(win(board, 'X'))
        io.writeln('X won!')
    elseif(win(board, 'O'))
        io.writeln('O won!')
    elseif(full(board))
        io.writeln('Draw!')
    else
        io.writeln('')
        io.writeln(str('turn '+turn))
        move(board, p1, board2)
        draw(board2)
        game(board2, p2, p1, turn+1)
```

From now on, we proceed with the main function of the game.

It's important to declare a `function` whenever,

- Procedural code is used. `io` and `throw` is procedural code.
- It's the main function of a procedural program, and thus doesn't need relational behaviour (i.e. the *game* function).

In addition, we avoid any complications that may arise from the use of relational negation or relational if-else. Although, if we're using conditionals procedurally as we're doing, it's already not relational code.

On the other hand, we managed to contain the procedural parts to a few functions: *game* which is essentially our main function and the player. This is the classical approach to making a declarative program, as even if you're making a declarative program, a lot of code you have to use might be procedural. That is, you make most of the program code declarative but call it through a procedural main function that uses I/O, etc. so as to interface the program with the user.

```
p1={
    piece='X'
    ai=_default
}
p2={
    piece='O'
    ai=_player//_default
}

board=_board
game(board, p1, p2, 1)
```

Finally, we define the players.

You may see the full code at the URL: <https://pastebin.com/n1ByALVd>

Adding UI

One may want to expand the program to use a graphical user-interface, such as by using the [Space](#) framework.

Exploring logic programming further

If we want to explore logic programming, one may set both AIs to `_default`.

```
rel pause()
    print('> ')
    c::ioread(x)
...
p2={
    piece='0'
    ai=_default
}
...
game(board,p1,p2,1)
pause()
false //the program will fail and search other possible moves
```

Simply adding `false` to the end will make the program fail, thus search for other ways the game could've gone. The AIs should extensively go through *every* possible game of tic-tac-toe.

In the URL, we've made even this impossible and the program will simply fail, as we've added the line.

```
once move(board,p1,board2)
```

`move` is *correctly* set to run once regardless of the non-deterministic AI in order to better work as a procedural function.